INSTITUT DU DÉVELOPPEMENT
ET DES RESSOURCES
EN INFORMATIQUE SCIENTIFIQUE

CENTRE NATIONAL
DE LA RECHERCHE
SCIENTIFIQUE

Transformées de Fourier rapides monoprocesseur sur NEC SX-5

Jalel Chergui
CNRS/IDRIS, Bâtiment 506, BP 167
91403 Orsay cedex, France
Jalel.Chergui@idris.fr

2 octobre 2000

Résumé

Dans cet article, nous présentons deux types de sous-programmes monoprocesseur très usuels de transformée de Fourier rapide complexe-complexe à une et trois dimensions, des bibliothèques ASL, CRAYFFT et JMFFT installées sur la grappe de machines NEC SX-5 de l'IDRIS. Nous comparons leurs performances ainsi que leur précision tout en mettant l'accent sur les paramètres susceptibles d'accélérer leur exécution sur ce type de machine.

1 Introduction

L'importance des TFR[1] dans le domaine du calcul numérique n'est certainement plus à démontrer et encore moins celle d'en disposer d'extrêmement performantes sur une architecture donnée de machine.
Avant la découverte des premiers algorithmes de TFR, on réalisait une Transformée de Fourier Discrète (TFD) de longueur N avec des coûts estimés à $\mathcal{O}(N^2)$ opérations. Elle s'avère très pénalisante comparée aux algorithmes de TFR apparus dans les années 60 et dont les coûts avoisinent $\mathcal{O}(N \log_2 N)$ opérations.
En effet, dès le milieu des années 60, la théorie des Transformées de Fourier Discrètes a connu une progression importante depuis que Cooley et Tukey [CJW] proposèrent leur algorithme de TFR. Cependant, en 1822 soit bien avant ces années-là, Fourier, pour résoudre l'équation de la chaleur, découvrit les propriétés sur les séries qui portent aujourd'hui son nom. Plus tard, en 1942, Danielson & Lanczos, montrèrent qu'une TFD de longueur N peut s'exprimer comme une somme de deux TFD de longueur $\frac{N}{2}$ chacune. Le plus intéressant dans l'algorithme de Danielson-Lanczos [DGC] est le fait que ce processus est applicable d'une façon récursive. Cette propriété fut justement utilisée par Cooley & Tukey. Chaque partie pouvant elle-même être décomposée en deux autres parties, le processus est répété $\frac{N}{2}$ fois. Plus généralement, à chaque étape $k = 1, \ldots, \frac{N}{2}$, chaque partie est décomposée en $\frac{N}{2\times k}$ transformées. Bien entendu, cela suppose que N soit une puissance de $b = 2$ ($N = b^r; r \in \mathbb{N}^+$). En réalité, l'algorithme est généralisable à une base quelconque $b \in \mathbb{N}_*^+$ et même à une combinaison de plusieurs bases (ex. $N = 2^3 \times 3^4 \times 5^7 \times \ldots$). En général, les bases qui permettent de reconstituer N sont choisies parmi les nombres premiers. Le cas le plus favorable est, bien entendu, celui pour lequel N est une puissance de 2. En revanche, le cas le plus défavorable

1. Nous adoptons ici la dénomination française désignant une « Transformée de Fourier Rapide » dont le pendant anglo-saxon est FFT, signifiant, bien entendu, *Fast Fourier Transform*.

(à moins de compléter le tableau à transformer par des zéros jusqu'à concurrence d'une taille en puissance de 2) se produit quand $N \gg 1$ est lui-même un nombre premier [CMR]. Dans ce cas, le coût de la TFR tend vers celui d'une TFD classique à savoir $\mathcal{O}(N^2)$. Il est bien évident que nous touchons là un aspect algorithmique essentiel pouvant affecter significativement les performances d'une TFR. Les ouvrages sur le sujet ne manquent pas. Pour une approche pratique, celui de Brigham [EOB] contenterait les plus pressés, alors que celui de Nussbaumer [HJN] présente un apport théorique suffisamment important pour satisfaire les plus curieux.
Au-delà des propriétés algorithmiques, les performances d'une TFR, sur une machine donnée, dépendent de son implémentation. Celle-ci doit tenir compte de l'architecture du processeur (scalaire ou vectoriel) et, essentiellement, de la hiérarchie mémoire (resp. mémoires caches ou registres vectoriels), afin de minimiser le côut des accès à la mémoire centrale. L'IDRIS, rappelons-le, dispose d'une grappe de trois nœuds `NEC SX-5`. Chaque nœud possède une architecture vectorielle multi-processeurs à mémoire partagée dont la performance maximale théorique par processeur est de 8 Gflops. Les bibliothèques de TFR monoprocesseur installées sur chacun des nœuds ont été spécialement vectorisées pour ce type d'architecture. Néanmoins, ainsi que nous le constaterons dans le paragraphe 3, les applications utilisant ces bibliothèques ne délivrent leurs performances nominales que lorsque la longueur moyenne des vecteurs dépasse largement la taille d'un registre vectoriel (ici 256 éléments) pour une occupation optimale des unités fonctionnelles, et qu'au moins la première dimension déclarée des tableaux est de longueur impaire, afin de minimiser les conflits de bancs lors des accès à la mémoire centrale. Cependant, nous constaterons que malgré toutes ces précautions, il existe une disparité des performances entre ces différentes bibliothèques. Dans le paragraphe suivant, nous indiquerons toutes celles disponibles sur la plate-forme indiquée ainsi que la façon d'utiliser certaines d'entres elles.

2 Disponibilité et utilisation

Sur les machines `NEC SX-5` de l'IDRIS, on trouve des sous-programmes de TFR dans les bibliothèques `ASL`, `CRAYFFT`, `JMFFT`, `NAG` et `IMSL`. Les deux dernières sont des bibliothèques commerciales bien connues dans le monde scientifique. Nous nous pencherons plus particulièrement sur les trois premières, à savoir :

i) `ASL` (*Advanced Scientific Library*) est une bibliothèque maintenue par NEC [ASL]. Elle concerne aussi bien les TFR que la résolution de problèmes d'algèbre linéaire et de statistique. Elle existe en mode I4 (entiers représentés sur 4 octets) et I8 (entiers représentés sur 8 octets) avec une représentation des nombres flottants en simple (R4), double (R8) et quadruple (R16) précision. À l'heure actuelle, c'est la bibliothèque de référence en terme de fonctionnalités et de performances. Outre les sous-programmes monoprocesseur, cette bibliothèque inclut leurs homologues parallèles multitâches.

ii) `CRAYFFT` est une bibliothèque de TFR maintenue par NEC [MAT]. C'est une émulation des sous-programmes de TFR CRAY sans être tout à fait conforme quant à la taille du tableau de travail et du tableau des coefficients trigonométriques. Elle existe en mode I4 et I8 avec une représentation des nombres flottants en simple (R4), double (R8) et quadruple (R16) précision. C'est une bibliothèque en cours d'évolution. Elle ne permet, à l'heure actuelle, que le calcul des TFR dont la taille des transformées est une combinaison des nombres premiers 2, 3 et 5.

iii) `JMFFT` est une bibliothèque de TFR développée à l'IDRIS par Jean-Marie Teuler [JMT]. C'est une émulation parfaitement compatible des sous-programmes de TFR monoprocesseur contenus dans la bibliothèque `SciLib` de CRAY. Elle existe en mode I4 et I8 avec une représentation des nombres flottants en double précision (R8) uniquement. Outre ses bonnes performances, l'accès aux sources Fortran et C est public.

2.1 Utilisation à l'édition de liens

Le tableau 1 indique le type des nombres entiers et la précision flottante admis par chacune des bibliothèques que nous venons de citer. Noter que ces bibliothèques peuvent toutes être utilisées avec une représentation des nombres entiers sur 8 octets (I8) conjointement à une double précision flottante (R8) [2].

Bibliothèques \ Types		`INTEGER*4` (I4)	`INTEGER*8` (I8)	`REAL*4` (R4)	`REAL*8` (R8)
`ASL`	`libasl.a`	✓		✓	✓
	`libasl64.a`		✓	✓	✓
`CRAYFFT`	`libfft.a`	✓		✓	✓
	`libfft_64.a`		✓		✓
`JMFFT`	`libjmfft_i4r8.a`	✓			✓
	`libjmfft_i8r8.a`		✓		✓

Tab. 1 – *Nombres entiers et précisions flottantes admis pour chacune des bibliothèques.*

Afin de faciliter l'utilisation de ces bibliothèques au chargement des programmes, il nous a semblé utile, à l'IDRIS, de définir une variable d'environnement pour chacune d'elles. Ainsi, par exemple, comme il existe deux bibliothèques `ASL` (tableau 1) ayant chacune son mode de représentation des nombres entiers, il a fallu définir deux variables d'environnement `$ASL_I4R4R8` et `$ASL_I8R8` contenant comme valeur respectivement les options `-lasl` et `-lasl64`.
Le tableau 2 résume l'ensemble de ces variables d'environnement et la façon de les utiliser à l'édition de liens. Fondamentalement, selon que les variables entières sont déclarées sur 4 ou sur 8 octets dans le programme, nous utiliserons les variables d'environnement suffixées respectivement par la chaîne `_I4` ou `_I8` à l'édition de liens. En plus, selon que les variables réelles sont déclarées en simple ou en double précision, nous utiliserons les variables d'environnement suffixées respectivement par la chaîne `_I4R4` ou `_I8R8`. Toutes les combinaisons ne sont pas pourvues. Ainsi des variables d'environnement suffixées par `_I8R4R8` n'existent pas. Dans ce tableau, il est indiqué en particulier l'utilisation de l'option `-ew` pour convertir, le cas échéant, les variables et les constantes entières et réelles sur 8 octets. Si cette option est utilisée, elle doit être spécifiée aussi bien à la compilation qu'à l'édition de liens pour que cette transformation soit automatiquement prise en charge. En revanche, il n'est pas obligatoire de l'utiliser si les variables entières et réelles sont explicitement déclarées respectivement du type `INTEGER(KIND=8)` et `REAL(KIND=8)` dans le programme.

2. Le mode `I8R8` intéresse tout particulièrement les utilisateurs venant de machines CRAY.

Emplacement	Modes	Édition de liens pour Fortran
/usr/lib/libasl.a	32 bits	`f90 mon_prog.o $ASL_I4R4R8`
/usr/lib/libasl64.a	64 bits	`f90 -ew mon_prog.o $ASL_I8R8`
/usr/lib/libfft.a	32 bits	`f90 mon_prog.o $CRAYFFT_I4R4R8`
/usr/lib/libfft_64.a	64 bits	`f90 -ew mon_prog.o $CRAYFFT_I8R8`
/usr/local/lib/libjmfft_i4r8.a	32 bits	`f90 mon_prog.o $JMFFT_I4R8`
/usr/local/lib/libjmfft_i8r8.a	64 bits	`f90 -ew mon_prog.o $JMFFT_I8R8`

TAB. 2 – *Utilisation des variables d'environnement associées aux bibliothèques de TFR pour l'édition de liens de modules objets Fortran.*

2.2 Utilisation dans un programme Fortran

À titre d'exemple, nous présentons, dans les figures 1, 2 et 3, trois programmes réalisant chacun une TFR complexe-complexe 3D directe et inverse d'un tableau `X`, respectivement avec `ASL` (figure 1, lignes 29 et 33), `CRAYFFT` (figure 2, lignes 27 et 32) et `JMFFT` (figure 3, lignes 31 et 36) en mode I8. Pour un tableau `X` donné, ces trois programmes doivent fournir un résultat identique à une précision de calcul près que l'on explicitera mieux dans le paragraphe 4. Notons que les sous-programmes de `ASL` prévoient un seul argument pour calculer la TFR du tableau `X` qui doit donc obligatoirement être à la fois en entrée et en sortie, contrairement aux autres bibliothèques où il est prévu deux arguments. Une différence remarquable se trouve dans la taille `nwork` du tableau de travail `work` qui est nettement plus importante pour `ASL` ou `CRAYFFT` comparée à celle nécessaire pour `JMFFT` (lignes 12 et 13 des figures 1, 2 et 3). Il faut noter cependant que les performances des TFR 2D et 3D de `JMFFT` sont paramètrées en fonction d'un facteur de blocage dont la taille par défaut est de 128 éléments. Celle-ci fait référence à la taille d'un registre vectoriel du `CRAY C98`. Sur le `NEC SX-5`, la taille d'un registre est deux fois plus importante ce qui nous a amené à augmenter la taille par défaut du tableau de travail `work` par l'effet conjoint de la constante `vlen` (ligne 12 de la figure 3) et du sous-programme auxiliaire `JMSETNWORK` (ligne 23 de la figure 3) de la bibliothèque `JMFFT`. Cette manipulation permet, quand la taille des TFR est supérieure à 256, de doubler les performances par rapport à celles obtenues avec une valeur de `vlen=128`.
De même, nous constatons une différence relativement appréciable, mais dans une moindre mesure que pour le tableau `work`, dans la taille `ntrigs` du tableau `trigs` utilisé pour le stockage des coefficients trigonométriques (lignes 9 et 10 des figures 1, 2 et 3).
Pour le calcul de la TFR inverse avec `ASL` (figure 1, ligne 33), remarquons que, contrairement aux autres bibliothèques, il faut explicitement appliquer le facteur d'échelle `scale` (ligne 35 de la figure 1) au résultat fourni, ici, par le sous-programme `LZFC3FB`[3].
Sur un tout autre sujet, rappelons qu'il n'est pas du tout anodin que les dimensions déclarées des tableaux (ex. lignes 4 des figures 1, 2 et 3) soient impaires. Il est bien connu que le partitionnement de la mémoire centrale en « bancs mémoires » provoque des conflits d'accès sitôt que deux éléments adressés consécutivement se trouvent sur le même banc mémoire. Dans une telle situation, le deuxième élément ne sera lu qu'une fois que ce banc aura été libéré suite à l'accès au premier élément. La dégradation en performance est alors d'autant plus importante que le pas d'adressage est en puissance de 2 et que celle-ci est élevée. Le cas typique se produit en Fortran dans le cas d'un accès consécutif à deux éléments d'une même

3. L'existence ou non de la première lettre (ici `L`) indique s'il s'agit d'un sous-programme du type I8 ou I4.

```
1  PROGRAM asl3d
2    IMPLICIT NONE
3
4    INTEGER, PARAMETER :: ldx=627, ldy=ldx, ldz=ldx
5    COMPLEX, DIMENSION(0:ldx-1,0:ldy-1,0:ldz-1) :: x  !... Tableau à transformer
6
7    INTEGER, PARAMETER :: n=5**4, m=n, l=n !... Taille des transformées
8
9    INTEGER, PARAMETER      :: ntrigs = 2*(n+m+l)
10   REAL, DIMENSION(ntrigs) :: trigs !... Tableau des coefficients trigonométriques
11
12   INTEGER, PARAMETER        :: nwork = ldx*ldy*ldz
13   COMPLEX, DIMENSION(nwork) :: work !... Tableau de travail
14
15   INTEGER, DIMENSION(60) :: ifax !... Tableau des facteurs premiers
16
17   INTEGER :: isign, ierr=0 !... Sens de la transformée et code d'erreur
18
19   REAL :: scale !... Facteur d'échelle
20
21   READ(10) x !... Initialisation du tableau x
22
23   !... Initialisation de la table trigonométrique
24   isign=0
25   CALL LZFC3FB(n, m, l, x, ldx, ldy, ldz, isign, ifax, trigs, work, ierr)
26
27   !... Calcul de la TFR
28   isign = -1
29   CALL LZFC3BF(n, m, l, x, ldx, ldy, ldz, isign, ifax, trigs, work, ierr)
30
31   !... Calcul de la TFR inverse
32   isign = 1
33   CALL LZFC3FB(n, m, l, x, ldx, ldy, ldz, isign, ifax, trigs, work, ierr)
34   scale = 1./REAL(n*m*l)
35   x(0:n-1,0:m-1,0:l-1) = x(0:n-1,0:m-1,0:l-1) * scale
36
37   PRINT *, x(0:n-1,1,1), x(0:n-1,m,l) !... Impression partielle du résultat
38 END PROGRAM asl3d
39
40 Compilation      : f90 -ew -c asl3d.f90
41 Édition de liens : f90 -ew asl3d.o $ASL_I8R8
```

FIG. 1 – *Programme Fortran calculant une TFR directe et inverse avec* ***ASL****.*

ligne d'une matrice. Alors, le pas d'adressage est implicite et égal à la première dimension de la matrice. Si celui-ci est paire, les conflits de bancs potentiels ainsi engendrés pourront conduire à une forte dégradation des performances.
Notons enfin que l'appel aux sous-programmes de `CRAYFFT` (lignes 23-32, figure 2) et de `JMFFT` (lignes 27-36, figure 3) sont syntaxiquement identiques en mode I8 tout en conservant une interface unique au niveau des arguments, puisque leur but commun, en plus de celui de fournir de bonnes performances, est d'émuler les TFR de la `SciLib` de CRAY.
Des programmes similaires à ceux que nous venons de commenter ont servi aux mesures de performances dont les résultats seront présentés dans le paragraphe suivant.

3 Performances

Nous allons tout particulièrement nous intéresser aux performances fournies par deux sous-programmes de TFR complexe-complexe 1D et 3D en mode `I8R8`. Les performances de ces deux sous-programmes semblent, en effet, être assez représentatives de l'ensemble des sous-programmes des bibliothèques en question. Les mesures que nous avons effectuées ne tiennent

```
1  PROGRAM crayfft3d
2    IMPLICIT NONE
3
4    INTEGER, PARAMETER :: ldx=627, ldy=ldx, ldz=ldx
5    COMPLEX, DIMENSION(0:ldx-1,0:ldy-1,0:ldz-1) :: x !... Tableau à transformer
6
7    INTEGER, PARAMETER :: n=5**4, m=n, l=n !... Taille des transformées
8
9    INTEGER, PARAMETER      :: ntrigs = 2*(n+m+l)
10   REAL, DIMENSION(ntrigs) :: trigs !... Tableau des coefficients trigonométriques
11
12   INTEGER, PARAMETER     :: nwork = 6*n*m*l
13   REAL, DIMENSION(nwork) :: work !... Tableau de travail
14
15   INTEGER :: isign, ierr=0 !... Sens de la transformée et code d'erreur
16
17   REAL :: scale=1. !... Facteur d'échelle
18
19   READ(10) x !... Initialisation du tableau x
20
21   !... Initialisation de la table trigonométrique
22   isign=0
23   CALL CCFFT3D(isign,n,m,l,scale,x,ldx,ldy,x,ldx,ldy,trigs,work,ierr)
24
25   !... Calcul de la TFR
26   isign = 1
27   CALL CCFFT3D(isign,n,m,l,scale,x,ldx,ldy,x,ldx,ldy,trigs,work,ierr)
28
29   !... Calcul de la TFR inverse
30   scale = 1./REAL(n*m*l)
31   isign = -1
32   CALL CCFFT3D(isign,n,m,l,scale,x,ldx,ldy,x,ldx,ldy,trigs,work,ierr)
33
34   PRINT *, x(0:n-1,1,1), x(0:n-1,m,l) !... Impression partielle du résultat
35 END PROGRAM crayfft3d
36
37 Compilation      : f90 -ew -c crayfft3d.f90
38 Édition de liens : f90 -ew crayfft3d.o $CRAYFFT_I8R8
```

FIG. 2 – *Programme Fortran calculant une TFR directe et inverse avec* `CRAYFFT`.

compte que de la TFR directe (ligne 29 des figures 1, 2 et 3) en négligeant par conséquent le temps d'initialisation des coefficients trigonométriques (ligne 25 des figures 1, 2 et 3). Le temps CPU est mesuré en millisecondes avec le sous-programme `clock`[4] dont la précision est estimée à une microseconde. Par ailleurs, outre les temps d'exécution, les performances flottantes des TFR sont fournies en termes de nombre d'opérations arithmétiques flottantes par seconde dont l'unité est le `Mflops` (1 Mflops=10^6 opérations flottantes par seconde).
Le compilateur Fortran utilisé est celui correspondant à la version 186 datant du 15 mars 2000. Quant aux numéros de version des bibliothèques, celui de `JMFFT` correspond au numéro 7.2, de `ASL` au numéro 15.1 et de `CRAYFFT` au numéro 1.0.
Sur les tableaux 3 et 4 sont reportées les mesures en temps CPU d'exécution ainsi que le nombre d'opérations flottantes par seconde respectivement des sous-programmes 1D et 3D relatifs aux trois bibliothèques en question.
En général, les performances sont nettement en faveur de `ASL` sauf dans le cas des TFR 1D où l'on constate que les performances de `JMFFT` sont meilleures en base 2 quand la taille N devient suffisamment importante (tableau 3) et demeurent, en outre, nettement au-dessus de celles de `CRAYFFT` en base 3 et 5.

4. Pour mesurer le temps CPU, la norme Fortran 95 définit le sous-programme `CPU_TIME`, non disponible sur NEC au moment où ces mesures ont été réalisées.

```
 PROGRAM jmfft3d
   IMPLICIT NONE

   INTEGER, PARAMETER :: ldx=627, ldy=ldx, ldz=ldx
   COMPLEX, DIMENSION(0:ldx-1,0:ldy-1,0:ldz-1) :: x !... Tableaux à transformer

   INTEGER, PARAMETER :: n=5**4, m=n, l=n !... Taille des transformées

   INTEGER, PARAMETER      :: ntrigs = 100+2*(n+m+l)
   REAL, DIMENSION(ntrigs) :: trigs !... Tableau des coefficients trigonométriques

   INTEGER, PARAMETER     :: vlen=256, nwork = 4*vlen*MAX(n, m, l)
   REAL, DIMENSION(nwork) :: work !... Tableau de travail

   INTEGER :: isign, ierr=0 !... Sens de la transformée et code d'erreur

   REAL :: scale=1. !... Facteur d'échelle

   READ(10) x !... Initialisation du tableau x

   !... Appel à un sous-programme auxiliaire de JMFFT pour modifier
   !... la taille par défaut du tableau de travail work au sein de JMFFT
   CALL JMSETNWORK(nwork)

   !... Initialisation de la table trigonométrique
   isign=0
   CALL CCFFT3D(isign,n,m,l,scale,x,ldx,ldy,x,ldx,ldy,trigs,work,ierr)

   !... Calcul de la TFR
   isign = 1
   CALL CCFFT3D(isign,n,m,l,scale,x,ldx,ldy,x,ldx,ldy,trigs,work,ierr)

   !... Calcul de la TFR inverse
   scale = 1./REAL(n*m*l)
   isign = -1
   CALL CCFFT3D(isign,n,m,l,scale,x,ldx,ldy,x,ldx,ldy,trigs,work,ierr)

   PRINT *, x(0:n-1,1,1), x(0:n-1,m,l) !... Impression partielle du résultat
 END PROGRAM jmfft3d

Compilation      : f90 -ew -c jmfft3d.f90
Édition de liens : f90 -ew jmfft3d.o $JMFFT_I8R8
```

FIG. 3 – *Programme Fortran calculant une TFR directe et inverse avec JMFFT.*

TFR C-C 1D	N=2^8	2^{10}	2^{15}	2^{20}	3^5	3^7	3^{10}	3^{13}	5^4	5^6	5^8	5^{10}
ASL (ms)	0,08	0,14	3,2	112	0,05	0,11	1,2	41	0,07	0,32	7,8	223
CRAYFFT (ms)	0,12	0,37	13,4	540	0,12	0,75	24,0	668	0,23	5,55	148,3	3792
JMFFT (ms)	0,88	0,92	2,3	61	0,91	0,98	3,1	70	0,94	1,87	28,4	771
ASL (Mflops)	88	263	559	706	161	1155	4532	4721	423	3600	5356	5688
CRAYFFT (Mflops)	61	104	154	165	86	178	217	277	134	216	273	342
JMFFT (Mflops)	12	55	1000	1550	17	185	2107	3183	88	1652	3557	4088

TAB. 3 – *Performances d'une TFR complexe-complexe 1D de taille N.*

Dans le cas de la TFR 3D (tableau 4), comme pour la TFR 1D, les performances de JMFFT en base 2 sont meilleures que celles de CRAYFFT. En revanche, la tendance s'inverse en base 3 et 5 où les performances de CRAYFFT l'emportent sur celles de JMFFT. À noter, cependant, que CRAYFFT occupe plus de place en mémoire que ses deux homologues puisqu'une réservation mémoire de 30 Go ne suffit pas à traiter une TFR de taille $(5^4)^3$ (dernière colonne du tableau 4) alors que celle-ci suffit largement pour ASL et JMFFT. Par ailleurs, il peut sembler paradoxal

TFR C-C 3D	N=2^4	2^6	2^8	2^9	3^2	3^3	3^4	3^5	5^1	5^2	5^3	5^4
`ASL` (ms)	0,12	3,2	285	2539	0,08	0,3	9	311	0,06	0,3	39	6248
`CRAYFFT` (ms)	0,50	36,3	1467	13001	0,17	1,0	14	351	0,06	0,8	48	?
`JMFFT` (ms)	1,37	12,7	473	4037	1,06	2,6	28	602	1,01	2,5	119	15054
`ASL` (Mflops)	1361	4710	4802	4931	355	3882	5540	5685	53	3677	5420	5768
`CRAYFFT` (Mflops)	294	440	959	1055	177	1360	3804	5208	53	1197	4317	?
`JMFFT` (Mflops)	163	1681	3757	3996	41	695	2344	3560	12	1207	4762	6259

TAB. 4 – *Performances d'une TFR complexe-complexe 3D de taille N^3.*

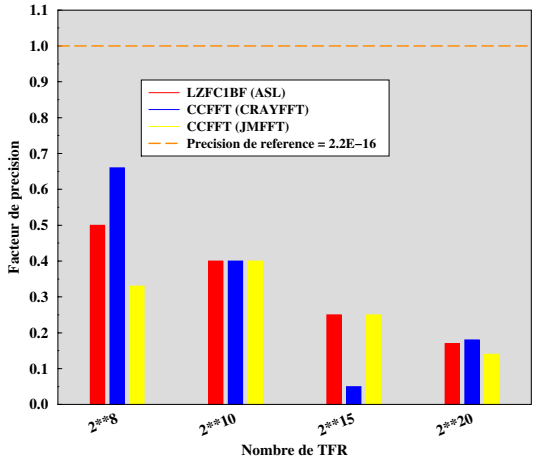
que dans ce cas `ASL` soit 2,5 fois plus rapide que `JMFFT` en temps d'exécution, alors que les performances de `JMFFT`, en terme de `Mflops`, sont légèrement supérieures. L'explication réside naturellement dans le fait que `JMFFT` exécute plus d'opérations flottantes que `ASL` pour résoudre un même problème. Cela montre, en général, que les deux mesures (temps et `Mflops`) sont nécessaires pour comparer la performance des applications sur une machine donnée.
Si nous nous limitons aux cas les plus gourmands en mémoire (dernière colonne des tableaux 3 et 4), les performances en termes de nombre d'opérations flottantes par seconde sont, à l'exception de celles de `CRAYFFT` en base 2, très honorables ($\sim$ 5000 `Mflops`) comparées à la performance crête d'un processeur qui est de 8000 `Mflops`.
Les performances relativement faibles observées dans le cas 1D (tableau 3) et 3D (tableau 4) lorsque la valeur de N est relativement petite s'expliquent, en partie, par le fait que la longueur moyenne des vecteurs est relativement courte compte tenu de l'architecture mémoire de la machine. Celle-ci fait que le temps d'utilisation des unités fonctionnelles ne permet pas de couvrir le temps de latence, relativement élevé, liée aux chargement des données de la mémoire vers les registres vectoriels sur le `SX-5`.
Dans le cas 3D, une optimisation monoprocesseur possible consiste à utiliser la TFR 1D en ayant pris soin au préalable de linéariser le tableau, ce qui permet d'accroître la longueur moyenne des vecteurs. L'utilisation de la version multitâches de `ASL` est aussi une option d'optimisation à envisager pour paralléliser une TFR multidimensionnelle.

4 Précision

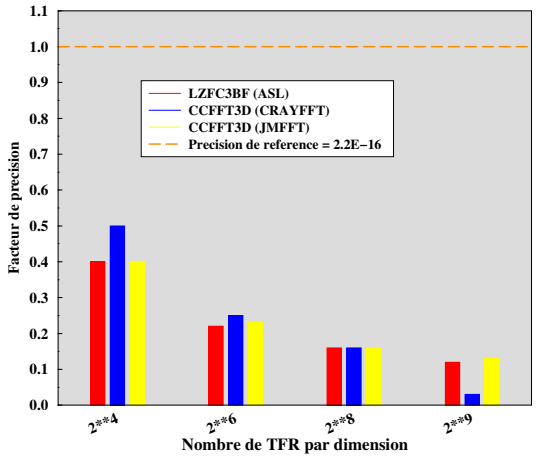
En théorie, il est bien établi que la TFR inverse $\tilde{x}$ de la TFR directe d'un tableau x est l'identité. Sur une machine donnée, cette proposition est vraie à une erreur d'arrondi près. Cette erreur, bien que relativement faible, varie non seulement selon la précision flottante du calcul mais aussi selon le nombre et l'agencement des opérations flottantes et des instructions exécutées dans un algorithme donné. Cette erreur d'arrondi sera comparée à une précision flottante de référence caractéristique de la machine, correspondant à la double précision flottante, retournée par la fonction Fortran 90 `EPSILON`. En réalité, cette fonction retourne l'écart absolu p entre l'unité et le nombre le plus proche en machine. Sur le `NEC SX-5`, cet écart vaut environ $p = 2 \times 10^{-16}$. Afin de normaliser l'erreur de calcul par rapport à la précision de référence p, nous définissons un facteur de précision $\alpha \in]0,1]$ tel que :

$$\alpha = \frac{p}{\|\tilde{x} - x\|}$$

Nous constaterons que l'erreur de calcul, définie ici par la norme $\|\tilde{x} - x\|$, reste minorée par

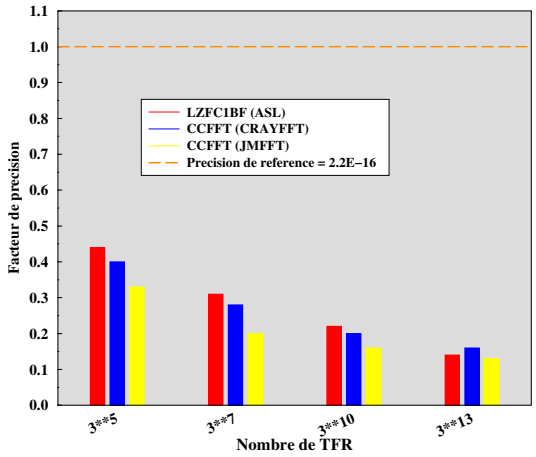


(a) TFR complexe-complexe 1D

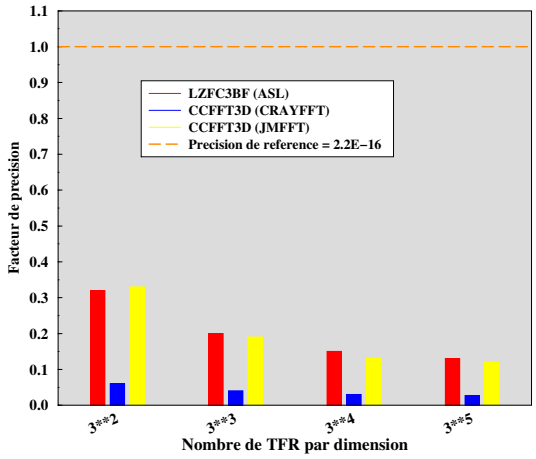


(b) TFR complexe-complexe 3D

FIG. 4 – *Facteur de précision des TFR en base 2*



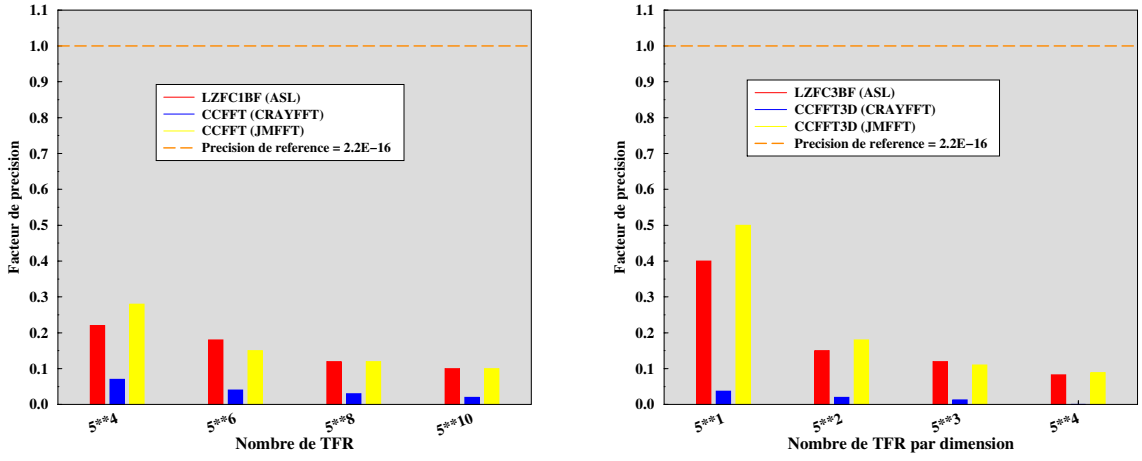
(a) TFR complexe-complexe 1D



(b) TFR complexe-complexe 3D

FIG. 5 – *Facteur de précision des TFR en base 3*

la précision de référence p. Ainsi, $\alpha \longrightarrow 1$ lorsque $\|\tilde{x} - x\| \longrightarrow p$. Alors, l'inverse de la TFR directe fournit une solution identique à celle d'origine à plus ou moins la précision p. Mais cela reste peu probable surtout en raison des algorithmes de contrôle de l'erreur d'arrondi et de la représentation discrète des nombres réels en machine.
Dans les figures 4, 5 et 6, sont représentées les variations du facteur de précision α associé aux calculs des TFR complexe-complexe 1D et 3D, respectivement en base 2, 3 et 5.
Globalement, le facteur de précision décroît en fonction de la taille de la TFR quelle que soit la bibliothèque utilisée. Cela peut s'expliquer par le fait que le nombre d'instructions flottantes exécuté croît forcément avec la taille d'une TFR, ce qui tend à augmenter les erreurs



(a) TFR complexe-complexe 1D

(b) TFR complexe-complexe 3D

FIG. 6 – *Facteur de précision des TFR en base 5*

d'arrondis. Cette décroissance est régulière et quasi-linéaire pour toutes les bibliothèques sauf pour les TFR 1D en base 2 de `JMFFT` et de `CRAYFFT` (figure 4-a) qui dérogent à cette règle. Les facteurs de précision de `ASL` et de `JMFFT` restent assez proches, contrairement à ceux de `CRAYFFT` en base 3 et 5 (figures 5-b et 6) où ils deviennent relativement bas en se situant entre 8 et 20 % de la précision fournie par `ASL` et `JMFFT`, dont la valeur avoisine 10^{-15} dans les cas observés où l'écart de précision est maximum.

5 Conclusion

Trois bibliothèques de transformées de Fourier rapides `ASL`, `JMFFT` et `CRAYFFT` ont été testées sur les machines `NEC SX-5` de l'IDRIS. Sans aucun doute, la bibliothèque la plus complète en terme de fonctionnalités, la plus performante et parmi les plus précises est aujourd'hui `ASL` qui est une bibliothèque sous licence NEC. La bibliothèque `JMFFT`, comme son homologue `CRAYFFT`, émule tous les sous-programmes de la bibliothèque `SciLib` de CRAY. En plus de ses performances honorables pour les TFR 1D, elle fournit une précision équivalente à `ASL`. La bibliothèque `CRAYFFT` est une bibliothèque sous licence NEC. Malgré la faible performance de ses TFR 1D, elle reste meilleure que `JMFFT` en base 3 et 5 pour les TFR multidimensionnelles. Sa précision de calcul est inférieure d'environ un ordre de grandeur comparée aux bibliothèques `ASL` et `JMFFT` dont la précision absolue avoisine 10^{-15} dans les cas observés les plus défavorables.

Références

[EOB] BRIGHAM, E.O., *The Fast Fourier Transform*. Englewood Cliffs, N.J.: Prentice-Hall, Chapter 13, 1974.

[CJW] COOLEY, J.W. & J.W. TUKEY, *An algorithm for the machine calculation of complex Fourier series*. Mathematics of Computation (1965), Vol. 19, No. 90, p. 297-301.

[DGC] Danielson, G.C. & C. Lanczos, *Some Improvements in Practical Fourier Analysis and their Application to X-Ray Scattering from Liquids*. J. Franklin Institute (April 1942), Vol. 233, p. 365-380, 435-452.

[HJN] Nussbaumer, H.J., *Fast Fourier Transform and Convolution Algorithms*. New York, Springer-Verlag, 1982.

[CMR] Rader, C.M., *Discrete Fourier Transforms when the Number of Data Points is Prime*. Proc. IEEE, IEEE Press, Piscataway, N.J., Vol.56, 1968, pp. 1107-1108.

[JMT] Bibliothèque développée par J.-M. Teuler. Émulation en Fortran et en C des TFR de la bibliothèque `SciLib` de CRAY : `<http://www.idris.fr/data/publications/JMFFT/JMFFT.html>`

[ASL] *Advanced Scientific Library*. Documentation NEC : `<http://www.idris.fr/users/doc_nec-users/ASL15.1/idxpdf_e.htm>` (accès restreint aux utilisateurs de l'IDRIS).

[MAT] *MathKeisen Fast Fourier Transforms Library*. Documentation NEC disponible en ligne par la commande : `man fft`.